

```
// 传递谁先行的指示, 进行游戏
cout << ((play(first)) ? "sorry, you lost"
        : "congrats, you won") << endl;
cout << "play again? Enter 'yes' or 'no'" << endl;
} while (cin >> resp && resp[0] == 'y');
```

我们用一个 do while 循环（参见 5.4.4 节，第 169 页）来反复提示用户进行游戏。



由于引擎返回相同的随机数序列（参见 17.4.1 节，第 661 页），所以我们在循环外声明引擎对象。否则，每步循环都会创建一个新引擎，从而每步循环都会生成相同的值。类似的，分布对象也要保持状态，因此也应该在循环外定义。

在此程序中使用 `bernoulli_distribution` 的一个原因是它允许我们调整选择先行一方的概率：

```
bernoulli_distribution b(.55); // 给程序一个微小的优势
```

如果 `b` 定义如上，则程序有 55/45 的机会先行。

17.4.2 节练习

练习 17.31: 对于本节中的游戏程序，如果在 do 循环内定义 `b` 和 `e`，会发生什么？

练习 17.32: 如果我们在循环内定义 `resp`，会发生什么？

练习 17.33: 修改 11.3.6 节（第 392 页）中的单词转换程序，允许对一个给定单词有多种转换方式，每次随机选择一种进行实际转换。

17.5 IO 库再探

在第 8 章中我们介绍了 IO 库的基本结构及其最常用的部分。在本节中，我们将介绍三个更特殊的 IO 库特性：格式控制、未格式化 IO 和随机访问。

753 17.5.1 格式化输入与输出

除了条件状态外（参见 8.1.2 节，第 279 页），每个 `iostream` 对象还维护一个格式状态来控制 IO 如何格式化的细节。格式状态控制格式化的某些方面，如整型值是几进制、浮点值的精度、一个输出元素的宽度等。

标准库定义了一组操纵符（manipulator）（参见 1.2 节，第 6 页）来修改流的格式状态，如表 17.7 和表 17.8 所示。一个操纵符是一个函数或是一个对象，会影响流的状态，并能用作输入或输出运算符的运算对象。类似输入和输出运算符，操纵符也返回它所处理的流对象，因此我们可以在一条语句中组合操纵符和数据。

我们已经在程序中使用过一个操纵符——`endl`，我们将它“写”到输出流，就像它是一个值一样。但 `endl` 不是一个普通值，而是一个操作：它输出一个换行符并刷新缓冲区。

很多操纵符改变格式状态

操纵符用于两大类输出控制：控制数值的输出形式以及控制补白的数量和位置。大多数改变格式状态的操纵符都是设置/复原成对的；一个操纵符用来将格式状态设置为一个新值，而另一个用来将其复原，恢复为正常的默认格式。



当操纵符改变流的格式状态时，通常改变后的状态对所有后续 IO 都生效。

当我们有一组 IO 操作希望使用相同的格式时，操纵符对格式状态的改变是持久的这一特性很有用。实际上，一些程序会利用操纵符的这一特性对其所有输入或输出重置一个或多个格式规则的行为。在这种情况下，操纵符会改变流这一特性就是满足要求的了。

但是，很多程序（而且更重要的是，很多程序员）期望流的状态符合标准库正常的默认设置。在这些情况下，将流的状态置于一个非标准状态可能会导致错误。因此，通常最好在不再需要特殊格式时尽快将流恢复到默认状态。

控制布尔值的格式

754

操纵符改变对象的格式状态的一个例子是 `boolalpha` 操纵符。默认情况下，`bool` 值打印为 1 或 0。一个 `true` 值输出为整数 1，而 `false` 输出为 0。我们可以通过对流使用 `boolalpha` 操纵符来覆盖这种格式：

```
cout << "default bool values: " << true << " " << false
    << "\nalpha bool values: " << boolalpha
    << true << " " << false << endl;
```

执行这段程序会得到下面的结果：

```
default bool values: 1 0
alpha bool values: true false
```

一旦向 `cout` “写入”了 `boolalpha`，我们就改变了 `cout` 打印 `bool` 值的方式。后续打印 `bool` 值的操作都会打印 `true` 或 `false` 而非 1 或 0。

为了取消 `cout` 格式状态的改变，我们使用 `noboolalpha`：

```
bool bool_val = get_status();
cout << boolalpha    // 设置 cout 的内部状态
    << bool_val
    << noboolalpha;   // 将内部状态恢复为默认格式
```

本例中我们改变了 `bool` 值的格式，但只对 `bool_val` 的输出有效。一旦完成此值的打印，我们立即将流恢复到初始状态。

指定整型值的进制

默认情况下，整型值的输入输出使用十进制。我们可以使用操纵符 `hex`、`oct` 和 `dec` 将其改为十六进制、八进制或是改回十进制：

```
cout << "default: " << 20 << " " << 1024 << endl;
cout << "octal: " << oct << 20 << " " << 1024 << endl;
cout << "hex: " << hex << 20 << " " << 1024 << endl;
cout << "decimal: " << dec << 20 << " " << 1024 << endl;
```

当编译并执行这段程序时，会得到如下输出：

```
default: 20 1024
octal: 24 2000
hex: 14 400
decimal: 20 1024
```

注意，类似 `boolalpha`，这些操纵符也会改变格式状态。它们会影响下一个和随后所有的整型输出，直至另一个操纵符又改变了格式为止。



操纵符 `hex`、`oct` 和 `dec` 只影响整型运算对象，浮点值的表示形式不受影响。

755 在输出中指出进制

默认情况下，当我们打印出数值时，没有可见的线索指出使用的是几进制。例如，20 是十进制的 20 还是 16 的八进制表示？当我们按十进制打印数值时，打印结果会符合我们的期望。如果需要打印八进制值或十六进制值，应该使用 `showbase` 操纵符。当对流应用 `showbase` 操纵符时，会在输出结果中显示进制，它遵循与整型常量中指定进制相同的规范：

- 前导 `0x` 表示十六进制。
- 前导 `0` 表示八进制。
- 无前导字符串表示十进制。

我们可以使用 `showbase` 修改前一个程序：

```
cout << showbase; // 当打印整型值时显示进制
cout << "default: " << 20 << " " << 1024 << endl;
cout << "in octal: " << oct << 20 << " " << 1024 << endl;
cout << "in hex: " << hex << 20 << " " << 1024 << endl;
cout << "in decimal: " << dec << 20 << " " << 1024 << endl;
cout << noshowbase; // 恢复流状态
```

修改后的程序的输出会更清楚地表明底层值到底是什么：

```
default: 20 1024
in octal: 024 02000
in hex: 0x14 0x400
in decimal: 20 1024
```

操纵符 `noshowbase` 恢复 `cout` 的状态，从而不再显示整型值的进制。

默认情况下，十六进制值会以小写打印，前导字符也是小写的 `x`。我们可以通过使用 `uppercase` 操纵符来输出大写的 `X` 并将十六进制数字 `a-f` 以大写输出：

```
cout << uppercase << showbase << hex
    << "printed in hexadecimal: " << 20 << " " << 1024
    << nouppercase << noshowbase << dec << endl;
```

这条语句生成如下输出：

```
printed in hexadecimal: 0X14 0X400
```

我们使用了操纵符 `nouppercase`、`noshowbase` 和 `dec` 来重置流的状态。

控制浮点数格式

我们可以控制浮点数输出三个种格式：

- 以多高精度（多少个数字）打印浮点值
- 数值是打印为十六进制、定点十进制还是科学记数法形式
- 对于没有小数部分的浮点值是否打印小数点

756

默认情况下，浮点值按六位数字精度打印；如果浮点值没有小数部分，则不打印小数点；根据浮点数的值选择打印成定点十进制或科学记数法形式。标准库会选择一种可读性更好的格式：非常大和非常小的值打印为科学记数法形式，其他值打印为定点十进制形式。

指定打印精度

默认情况下，精度会控制打印的数字的总数。当打印时，浮点值按当前精度舍入而非截断。因此，如果当前精度为四位数字，则 3.14159 将打印为 3.142；如果精度为三位数字，则打印为 3.14。

我们可以通过调用 IO 对象的 precision 成员或使用 setprecision 操纵符来改变精度。precision 成员是重载的（参见 6.4 节，第 206 页）。一个版本接受一个 int 值，将精度设置为此值，并返回旧精度值。另一个版本不接受参数，返回当前精度值。setprecision 操纵符接受一个参数，用来设置精度。



操纵符 setprecision 和其他接受参数的操纵符都定义在头文件 iomanip 中。

下面的程序展示了控制浮点值打印精度的不同方法：

```
// cout.precision 返回当前精度值
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;
// cout.precision(12) 将打印精度设置为 12 位数字
cout.precision(12);
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;
// 另一种设置精度的方法是使用 setprecision 操纵符
cout << setprecision(3);
cout << "Precision: " << cout.precision()
    << ", Value: " << sqrt(2.0) << endl;
```

编译并执行这段程序，会得到如下输出：

```
Precision: 6, Value: 1.41421
Precision: 12, Value: 1.41421356237
Precision: 3, Value: 1.41
```

此程序调用标准库 sqrt 函数，它定义在头文件 cmath 中。sqrt 函数是重载的，不同版本分别接受一个 float、double 或 long double 参数，返回实参的平方根。

757

表 17.17: 定义在 iostream 中的操纵符	
boolalpha	将 true 和 false 输出为字符串
* noboolalpha	将 true 和 false 输出为 1, 0
showbase	对整型值输出表示进制的前缀
* noshowbase	不生成表示进制的前缀
showpoint	对浮点值总是显示小数点

续表

* noshowpoint	只有当浮点值包含小数部分时才显示小数点
showpos	对非负数显示+
* noshowpos	对非负数不显示+
uppercase	在十六进制值中打印 0x, 在科学记数法中打印 E
* nouppercase	在十六进制值中打印 0x, 在科学记数法中打印 e
* dec	整型值显示为十进制
hex	整型值显示为十六进制
oct	整型值显示为八进制
left	在值的右侧添加填充字符
right	在值的左侧添加填充字符
internal	在符号和值之间添加填充字符
fixed	浮点值显示为定点十进制
scientific	浮点值显示为科学记数法
hexfloat	浮点值显示为十六进制 (C++11 新特性)
defaultfloat	重置浮点数为格式为十进制 (C++11 新特性)
unitbuf	每次输出操作后都刷新缓冲区
* nounitbuf	恢复正常的缓冲区刷新方式
* skipws	输入运算符跳过空白符
noskipws	输入运算符不跳过空白符
flush	刷新 ostream 缓冲区
ends	插入空字符, 然后刷新 ostream 缓冲区
endl	插入换行, 然后刷新 ostream 缓冲区

* 表示默认流状态

指定浮点数记数法

Best Practices

除非你需要控制浮点数的表示形式 (如, 按列打印数据或打印表示金额或百分比的数据), 否则由标准库选择记数法是最好的方式。

通过使用恰当的操纵符, 我们可以强制一个流使用科学记数法、定点十进制或是十六进制记数法。操纵符 `scientific` 改变流的状态来使用科学记数法。操纵符 `fixed` 改变流的状态来使用定点十进制。

C++11

在新标准库中, 通过使用 `hexfloat` 也可以强制浮点数使用十六进制格式。新标准库还提供另一个名为 `defaultfloat` 的操纵符, 它将流恢复到默认状态——根据要打印的值选择记数法。

758

这些操纵符也会改变流的精度的默认含义。在执行 `scientific`、`fixed` 或 `hexfloat` 后, 精度值控制的是小数点后面的数字位数, 而默认情况下精度值指定的是数字的总位数——既包括小数点之后的数字也包括小数点之前的数字。使用 `fixed` 或 `scientific` 令我们可以按列打印数值, 因为小数点距小数部分的距离是固定的:

```
cout << "default format: " << 100 * sqrt(2.0) << '\n'
    << "scientific: " << scientific << 100 * sqrt(2.0) << '\n'
    << "fixed decimal: " << fixed << 100 * sqrt(2.0) << '\n'
    << "hexadecimal: " << hexfloat << 100 * sqrt(2.0) << '\n'
    << "use defaults: " << defaultfloat << 100 * sqrt(2.0)
```

```
<< "\n\n";
```

此程序会生成下面的输出：

```
default format: 141.421
scientific: 1.414214e+002
fixed decimal: 141.421356
hexadecimal: 0x1.1ad7bcp+7
use defaults: 141.421
```

默认情况下，十六进制数字和科学记数法中的 e 都打印成小写形式。我们可以用 uppercase 操纵符打印这些字母的大写形式。

打印小数点

默认情况下，当一个浮点值的小数部分为 0 时，不显示小数点。showpoint 操纵符强制打印小数点：

```
cout << 10.0 << endl;           // 打印 10
cout << showpoint << 10.0       // 打印 10.0000
    << noshowpoint << endl;    // 恢复小数点的默认格式
```

操纵符 noshowpoint 恢复默认行为。下一个输出表达式将有默认行为，即，当浮点值的小数部分为 0 时不输出小数点。

输出补白

当按列打印数据时，我们常常需要非常精细地控制数据格式。标准库提供了一些操纵符帮助我们完成所需的控制：

- setw 指定下一个数字或字符串值的最小空间。
- left 表示左对齐输出。
- right 表示右对齐输出，右对齐是默认格式。
- internal 控制负数的符号的位置，它左对齐符号，右对齐值，用空格填满所有中间空间。
- setfill 允许指定一个字符代替默认的空格来补白输出。

Note

setw 类似 endl，不改变输出流的内部状态。它只决定下一个输出的大小。

下面程序展示了如何使用这些操纵符：

```
int i = -16;
double d = 3.14159;
// 补白第一列，使用输出中最小 12 个位置
cout << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// 补白第一列，左对齐所有列
cout << left
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n'
    << right; // 恢复正常对齐
// 补白第一列，右对齐所有列
cout << right
    << "i: " << setw(12) << i << "next col" << '\n'
```

```
<< "d: " << setw(12) << d << "next col" << '\n';
// 补白第一列，但补在域的内部
cout << internal
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n';
// 补白第一列，用#作为补白字符
cout << setfill('#')
    << "i: " << setw(12) << i << "next col" << '\n'
    << "d: " << setw(12) << d << "next col" << '\n'
    << setfill(' '); // 恢复正常的补白字符
```

执行这段程序，会得到下面的输出：

```
i:          -16next col
d:         3.14159next col
i: -16          next col
d: 3.14159      next col
i:          -16next col
d:         3.14159next col
i: -          16next col
d:         3.14159next col
i: -#####16next col
d: #####3.14159next col
```

760

表 17.18: 定义在 `iomanip` 中的操纵符

<code>setfill(ch)</code>	用 <code>ch</code> 填充空白
<code>setprecision(n)</code>	将浮点精度设置为 <code>n</code>
<code>setw(w)</code>	读或写值的宽度为 <code>w</code> 个字符
<code>setbase(b)</code>	将整数输出为 <code>b</code> 进制

控制输入格式

默认情况下，输入运算符会忽略空白符（空格符、制表符、换行符、换纸符和回车符）。下面的循环

```
char ch;
while (cin >> ch)
    cout << ch;
```

当给定下面输入序列时

```
a b      c
d
```

循环会执行 4 次，读取字符 `a` 到 `d`，跳过中间的空格以及可能的制表符和换行符。此程序的输出是

```
abcd
```

操纵符 `noskipws` 会令输入运算符读取空白符，而不是跳过它们。为了恢复默认行为，我们可以使用 `skipws` 操纵符：

```
cin >> noskipws; // 设置 cin 读取空白符
while (cin >> ch)
    cout << ch;
```

```
cin >> skipws; // 将 cin 恢复到默认状态, 从而丢弃空白符
```

给定与前一个程序相同的输入, 此循环会执行 7 次, 从输入中既读取普通字符又读取空白符。此循环的输出为

```
a b      c
d
```

17.5.1 节练习

练习 17.34: 编写一个程序, 展示如何使用表 17.17 和表 17.18 中的每个操纵符。

练习 17.35: 修改第 670 页中的程序, 打印 2 的平方根, 但这次打印十六进制数字的大写形式。

练习 17.36: 修改上一题中的程序, 打印不同的浮点数, 使它们排成一列。

17.5.2 未格式化的输入/输出操作

761

到目前为止, 我们的程序只使用过**格式化 IO (formatted IO)**操作。输入和输出运算符 (<<和>>) 根据读取或写入的数据类型来格式化它们。输入运算符忽略空白符, 输出运算符应用补白、精度等规则。

标准库还提供了一组低层操作, 支持**未格式化 IO (unformatted IO)**。这些操作允许我们将一个流当作一个无解释的字节序列来处理。

单字节操作

有几个未格式化操作每次一个字节地处理流。这些操作列在表 17.19 中, 它们会读取而不是忽略空白符。例如, 我们可以使用未格式化 IO 操作 `get` 和 `put` 来读取和写入一个字符:

```
char ch;
while (cin.get(ch))
    cout.put(ch);
```

此程序保留输入中的空白符, 其输出与输入完全相同。它的执行过程与前一个使用 `noskipws` 的程序完全相同。

表 17.19: 单字节低层 IO 操作	
<code>is.get(ch)</code>	从 istream is 读取下一个字节存入字符 ch 中。返回 is
<code>os.put(ch)</code>	将字符 ch 输出到 ostream os。返回 os
<code>is.get()</code>	将 is 的下一个字节作为 int 返回
<code>is.putback(ch)</code>	将字符 ch 放回 is。返回 is
<code>is.unget()</code>	将 is 向后移动一个字节。返回 is
<code>is.peek()</code>	将下一个字节作为 int 返回, 但不从流中删除它

将字符放回输入流

有时我们需要读取一个字符才能知道还未准备好处理它。在这种情况下, 我们希望能将字符放回流中。标准库提供了三种方法退回字符, 它们有着细微的差别:

- `peek` 返回输入流中下一个字符的副本, 但不会将它从流中删除, `peek` 返回的值仍然留在流中。

- `unget` 使得输入流向后移动，从而最后读取的值又回到流中。即使我们不知道最后从流中读取什么值，仍然可以调用 `unget`。
- `putback` 是更特殊版本的 `unget`：它退回从流中读取的最后一个值，但它接受一个参数，此参数必须与最后读取的值相同。

762 一般情况下，在读取下一个值之前，标准库保证我们可以退回最多一个值。即，标准库不保证在中间不进行读取操作的情况下能连续调用 `putback` 或 `unget`。

从输入操作返回的 `int` 值

函数 `peek` 和无参的 `get` 版本都以 `int` 类型从输入流返回一个字符。这有些令人吃惊，可能这些函数返回一个 `char` 看起来会更自然。

这些函数返回一个 `int` 的原因是：可以返回文件尾标记。我们使用 `char` 范围中的每个值来表示一个真实字符，因此，取值范围中没有额外的值可以用来表示文件尾。

返回 `int` 的函数将它们要返回的字符先转换为 `unsigned char`，然后再将结果提升到 `int`。因此，即使字符集中有字符映射到负值，这些操作返回的 `int` 也是正值（参见 2.1.2 节，第 32 页）。而标准库使用负值表示文件尾，这样就可以保证与任何合法字符的值都不同。头文件 `cstdio` 定义了一个名为 `EOF` 的 `const`，我们可以用它来检测从 `get` 返回的值是否是文件尾，而不必记忆表示文件尾的实际数值。对我们来说重要的是，用一个 `int` 来保存从这些函数返回的值：

```
int ch; // 使用一个 int，而不是一个 char 来保存 get() 的返回值
// 循环读取并输出输入中的所有数据
while ((ch = cin.get()) != EOF)
    cout.put(ch);
```

此程序与第 673 页中的程序完成相同的工作，唯一的不同是用来读取输入的 `get` 版本不同。

多字节操作

一些未格式化 IO 操作一次处理大块数据。如果速度是要考虑的重点问题的话，这些操作是很重要的，但类似其他低层操作，这些操作也容易出错。特别是，这些操作要求我们自己分配并管理用来保存和提取数据的字符数组（参见 12.2 节，第 423 页）。表 17.20 列出了多字节操作。

表 17.20: 多字节低层 IO 操作
is.get(sink, size, delim) 从 is 中读取最多 size 个字节，并保存在字符数组中，字符数组的起始地址由 sink 给出。读取过程直至遇到字符 delim 或读取了 size 个字节或遇到文件尾时停止。如果遇到了 delim，则将其留在输入流中，不读取出来存入 sink is.getline(sink, size, delim) 与接受三个参数的 get 版本类似，但会读取并丢弃 delim is.read(sink, size) 读取最多 size 个字节，存入字符数组 sink 中。返回 is is.gcount() 返回上一个未格式化读取操作从 is 读取的字节数 os.write(source, size) 将字符数组 source 中的 size 个字节写入 os。返回 os

续表

```
is.ignore(size, delim)
```

读取并忽略最多 `size` 个字符，包括 `delim`。与其他未格式化函数不同，`ignore` 有默认参数：`size` 的默认值为 1，`delim` 的默认值为文件尾

`get` 和 `getline` 函数接受相同的参数，它们的行为类似但不相同。在两个函数中，`sink` 都是一个 `char` 数组，用来保存数据。两个函数都一直读取数据，直至下面条件之一发生：

- 已读取了 `size-1` 个字符
- 遇到了文件尾
- 遇到了分隔符

两个函数的差别是处理分隔符的方式：`get` 将分隔符留作 `istream` 中的下一个字符，而 `getline` 则读取并丢弃分隔符。无论哪个函数都不会将分隔符保存在 `sink` 中。



一个常见的错误是本想从流中删除分隔符，但却忘了做。

763

确定读取了多少个字符

某些操作从输入读取未知个数的字节。我们可以调用 `gcount` 来确定最后一个未格式化输入操作读取了多少个字符。应该在任何后续未格式化输入操作之前调用 `gcount`。特别是，将字符退回流的单字符操作也属于未格式化输入操作。如果在调用 `gcount` 之前调用了 `peek`、`unget` 或 `putback`，则 `gcount` 的返回值为 0。

小心：低层函数容易出错

一般情况下，我们主张使用标准库提供的高层抽象。返回 `int` 的 IO 操作很好地解释了原因。

一个常见的编程错误是将 `get` 或 `peek` 的返回值赋予一个 `char` 而不是一个 `int`。这样做是错误的，但编译器却不能发现这个错误。最终会发生什么依赖于程序运行于哪台机器以及输入数据是什么。例如，在一台 `char` 被实现为 `unsigned char` 的机器上，下面的循环永远不会停止：

```
char ch; // 此处使用 char 就是引入灾难！
// 从 cin.get 返回的值被转换为 char，然后与一个 int 比较
while ((ch = cin.get()) != EOF)
    cout.put(ch);
```

问题出在当 `get` 返回 `EOF` 时，此值会被转换为一个 `unsigned char`。转换得到的值与 `EOF` 的 `int` 值不再相等，因此循环永远也不会停止。这种错误很可能在调试时发现。

在一台 `char` 被实现为 `signed char` 的机器上，我们不能确定循环的行为。当一个越界的值被赋予一个 `signed` 变量时会发生什么完全取决于编译器。在很多机器上，这个循环可以正常工作，除非输入序列中有一个字符与 `EOF` 值匹配。虽然在普通数据中这种字符不太可能出现，但低层 IO 通常用于读取二进制值的场合，而这些二进制值不能直接映射到普通字符和数值。例如，在我们的机器上，如果输入中包含有一个值为 `'\377'` 的字符，则循环会提前终止。因为在我们的机器上，将 `-1` 转换为一个 `signed char`，就会得到 `'\377'`。如果输入中有这个值，则它会被（过早）当作文件尾指示符。

当我们读写有类型的值时，这种错误就不会发生。如果你可以使用标准库提供的类型更加安全、更高层的操作，就应该使用它们。

17.5.2 节练习

- 练习 17.37: 用未格式化版本的 `getline` 逐行读取一个文件。测试你的程序，给它一个文件，既包含空行又包含长度超过你传递给 `getline` 的字符数组大小的行。
- 练习 17.38: 扩展上一题中你的程序，将读入的每个单词打印到它所在的行。

17.5.3 流随机访问

各种流类型通常都支持对流中数据的随机访问。我们可以重定位流，使之跳过一些数据，首先读取最后一行，然后读取第一行，依此类推。标准库提供了一对函数，来定位(`seek`)到流中给定的位置，以及告诉(`tell`)我们当前位置。



随机 IO 本质上是依赖于系统的。为了理解如何使用这些特性，你必须查询系统文档。

764

虽然标准库为所有流类型都定义了 `seek` 和 `tell` 函数，但它们是否会做有意义的事情依赖于流绑定到哪个设备。在大多数系统中，绑定到 `cin`、`cout`、`cerr` 和 `clog` 的流不支持随机访问——毕竟，当我们向 `cout` 直接输出数据时，类似向回跳十个位置这种操作是没有意义的。对这些流我们可以调用 `seek` 和 `tell` 函数，但在运行时会出现，将流置于一个无效状态。



由于 `istream` 和 `ostream` 类型通常不支持随机访问，所以本节剩余内容只适用于 `fstream` 和 `sstream` 类型。

765

seek 和 tell 函数

为了支持随机访问，IO 类型维护一个标记来确定下一个读写操作要在哪里进行。它们还提供了两个函数：一个函数通过将标记 `seek` 到一个给定位置来重定位它；另一个函数 `tell` 我们标记的当前位置。标准库实际上定义了两对 `seek` 和 `tell` 函数，如表 17.21 所示。一对用于输入流，另一对用于输出流。输入和输出版本的差别在于名字的后缀是 `g` 还是 `p`。`g` 版本表示我们正在“获得”(读取)数据，而 `p` 版本表示我们正在“放置”(写入)数据。

表 17.21: seek 和 tell 函数	
<code>tellg()</code> <code>tellp()</code>	返回一个输入流中 (<code>tellg</code>) 或输出流中 (<code>tellp</code>) 标记的当前位置
<code>seekg(pos)</code> <code>seekp(pos)</code>	在一个输入流或输出流中将标记重定位到给定的绝对地址。 <code>pos</code> 通常是前一个 <code>tellg</code> 或 <code>tellp</code> 返回的值
<code>seekp(off, from)</code> <code>seekg(off, from)</code>	在一个输入流或输出流中将标记定位到 <code>from</code> 之前或之后 <code>off</code> 个字符， <code>from</code> 可以是下列值之一 • <code>beg</code>，偏移量相对于流开始位置 • <code>cur</code>，偏移量相对于流当前位置 • <code>end</code>，偏移量相对于流结尾位置

从逻辑上讲，我们只能对 `istream` 和派生自 `istream` 的类型 `ifstream` 和 `istringstream` (参见 8.1 节，第 278 页) 使用 `g` 版本，同样只能对 `ostream` 和派生自 `ostream` 的类型 `ofstream` 和 `ostringstream` 使用 `p` 版本。一个 `iostream`、

`fstream` 或 `stringstream` 既能读又能写关联的流，因此对这些类型的对象既能使用 `g` 版本又能使用 `p` 版本。

只有一个标记

标准库区分 `seek` 和 `tell` 函数的“放置”和“获得”版本这一特性可能会导致误解。即使标准库进行了区分，但它在一个流中只维护单一的标记——并不存在独立的读标记和写标记。

当我们处理一个只读或只写的流时，两种版本的区别甚至是不明显的。我们可以对这些流只使用 `g` 或只使用 `p` 版本。如果我们试图对一个 `ifstream` 流调用 `tellp`，编译器会报告错误。类似的，编译器也不允许我们对一个 `ostream` 调用 `seekg`。

`fstream` 和 `stringstream` 类型可以读写同一个流。在这些类型中，有单一的缓冲区用于保存读写的数据，同样，标记也只有一个，表示缓冲区中的当前位置。标准库将 `g` 和 `p` 版本的读写位置都映射到这个单一的标记。



由于只有单一的标记，因此只要我们在读写操作间切换，就必须进行 `seek` 操作来重定位标记。

766

重定位标记

`seek` 函数有两个版本：一个移动到文件中的“绝对”地址；另一个移动到一个给定位置的指定偏移量：

```
// 将标记移动到一个固定位置
seekg(new_position); // 将读标记移动到指定的 pos_type 类型的位置
seekp(new_position); // 将写标记移动到指定的 pos_type 类型的位置

// 移动到给定起始点之前或之后指定的偏移位置
seekg(offset, from); // 将读标记移动到距 from 偏移量为 offset 的位置
seekp(offset, from); // 将写标记移动到距 from 偏移量为 offset 的位置
```

`from` 的可能值如表 17.21 所示。

参数 `new_position` 和 `offset` 的类型分别是 `pos_type` 和 `off_type`，这两个类型都是机器相关的，它们定义在头文件 `istream` 和 `ostream` 中。`pos_type` 表示一个文件位置，而 `off_type` 表示距当前位置的一个偏移量。一个 `off_type` 类型的值可以是正的也可以是负的，即，我们可以在文件中向前移动或向后移动。

访问标记

函数 `tellg` 和 `tellp` 返回一个 `pos_type` 值，表示流的当前位置。`tell` 函数通常用来记住一个位置，以便稍后再定位回来：

```
// 记住当前写位置
ostream writeStr; // 输出 stringstream
ostream::pos_type mark = writeStr.tellp();
// ...
if (cancelEntry)
    // 回到刚才记住的位置
    writeStr.seekp(mark);
```

读写同一个文件

我们来考察一个编程实例。假定已经给定了一个要读取的文件，我们要在此文件的末尾写入新的一行，这一行包含文件中每行的相对起始位置。例如，给定下面文件：

```
abcd
efg
hi
j
```

程序应该生成如下修改过的文件：

767

```
abcd
efg
hi
j
5 9 12 14
```

注意，我们的程序不必输出第一行的偏移——它总是从位置 0 开始。还要注意，统计偏移量时必须包含每行末尾不可见的换行符。最后，注意输出的最后一个数是我们的输出开始那行的偏移量。在输出中包含了这些偏移量后，我们的输出就与文件的原始内容区分开来了。我们可以读取结果文件中最后一个数，定位到对应偏移量，即可得到我们的输出的起始地址。

我们的程序将逐行读取文件。对每一行，我们将递增计数器，将刚刚读取的一行的长度加到计数器上，则此计数器即为下一行的起始地址：

```
int main()
{
    // 以读写方式打开文件，并定位到文件尾
    // 文件模式参数参见 8.2.2 节（第 286 页）
    fstream inOut("copyOut",
                  fstream::ate | fstream::in | fstream::out);
    if (!inOut) {
        cerr << "Unable to open file!" << endl;
        return EXIT_FAILURE; // EXIT_FAILURE 参见 6.3.2 节（第 204 页）
    }
    // inOut 以 ate 模式打开，因此一开始就定义到其文件尾
    auto end_mark = inOut.tellg(); // 记住原文件尾位置
    inOut.seekg(0, fstream::beg); // 重定位到文件开始
    size_t cnt = 0; // 字节数累加器
    string line; // 保存输入中的每行
    // 继续读取的条件：还未遇到错误且还在读取原数据
    while (inOut && inOut.tellg() != end_mark
           && getline(inOut, line)) { // 且还可获取一行输入
        cnt += line.size() + 1; // 加 1 表示换行符
        auto mark = inOut.tellg(); // 记住读取位置
        inOut.seekp(0, fstream::end); // 将写标记移动到文件尾
        inOut << cnt; // 输出累计的长度
        // 如果不是最后一行，打印一个分隔符
        if (mark != end_mark) inOut << " ";
        inOut.seekg(mark); // 恢复读位置
    }
    inOut.seekp(0, fstream::end); // 定位到文件尾
```

```
    inOut << "\n"; // 在文件尾输出一个换行符
    return 0;
}
```

我们的程序用 `in`、`out` 和 `ate` 模式（参见 8.2.2 节，第 286 页）打开 `fstream`。前两个模式指出我们想读写同一个文件。指定 `ate` 会将读写标记定位到文件尾。与往常一样，我们检查文件是否成功打开，如果失败就退出（参见 6.3.2 节，第 203 页）。 768

由于我们的程序向输入文件写入数据，因此不能通过文件尾来判断是否停止读取，而是应该在达到原数据的末尾时停止。因此，我们必须首先记住原文件尾的位置。由于我们是以 `ate` 模式打开文件的，因此 `inOut` 已经定位到文件尾了。我们将当前位置（即，原文件尾）保存在 `end_mark` 中。记住文件尾位置之后，我们 `seek` 到距文件起始位置偏移量为 0 的地方，即，将读标记重定位到文件起始位置。

`while` 循环的条件由三部分组成：首先检查流是否合法；如果合法，通过比较当前位置（由 `tellg` 返回）和记录在 `end_mark` 中的位置来检查是否读完了原数据；最后，假定前两个检查都已成功，我们调用 `getline` 读取输入的下一行，如果 `getline` 成功，则执行 `while` 循环体。

循环体首先将当前位置记录在 `mark` 中。我们保存当前位置是为了在输出下一个偏移量后再退回来。接下来调用 `seekp` 将写标记重定位到文件尾。我们输出计数器的值，然后调用 `seekg` 回到记录在 `mark` 中的位置。回退到原位置后，我们就准备好继续检查循环条件了。

每步循环都会输出下一行的偏移量。因此，最后一步循环负责输出最后一行的偏移量。但是，我们还需要在文件尾输出一个换行符。与其他写操作一样，在输出换行符之前我们调用 `seekp` 来定位到文件尾。

17.5.3 节练习

练习 17.39: 对本节给出的 `seek` 程序，编写你自己的版本。

小结

本章介绍了一些特殊 IO 操作和四个标准库类型：tuple、bitset、正则表达式和随机数。

tuple 是一个模板，允许我们将多个不同类型的成员捆绑成单一对象。每个 tuple 包含指定数量的成员，但对一个给定的 tuple 类型，标准库并未限制我们可以定义的成员数量上限。

bitset 允许我们定义指定大小的二进制位集合。标准库不限制一个 bitset 的大小必须与整型类型的大小匹配，bitset 的大小可以更大。除了支持普通的位运算符（参见 4.8 节，第 136 页）外，bitset 还定义了一些命名的操作，允许我们操纵 bitset 中特定定位的状态。

正则表达式库提供了一组类和函数：regex 类管理用某种正则表达式语言编写的正则表达式。匹配类保存了某个特定匹配的相关信息。这些类被函数 regex_search 和 regex_match 所用。这两个函数接受一个 regex 对象和一个字符序列，检查 regex 中的正则表达式是否匹配给定的字符序列。regex 迭代器类型是迭代器适配器，它们使用 regex_search 遍历输入序列，返回每个匹配的子序列。标准库还定义了一个 regex_replace 函数，允许我们用指定内容替换输入序列中与正则表达式匹配的部分。

随机数库由一组随机数引擎类和分布类组成。随机数引擎返回一个均匀分布的整型值序列。标准库定义了多个引擎，它们具有不同的性能特点。default_random_engine 是适合于大多数普通情况的引擎。标准库还定义了 20 个分布类型。这些分布类型使用一个引擎来生成指定类型的随机数，这些随机数的值都在给定范围内，且分布满足指定的概率分布。

术语表

bitset 标准库类，保存二进制位集合，大小在编译时已知，并提供检测和设置集合中二进制位的操作。

cmatch **csub_match** 对象的容器，保存一个 regex 与一个 const char* 输入序列匹配的相关信息。容器首元素描述了整个匹配结果。后续元素描述了子表达式的匹配结果。

cregex_iterator 类似 sregex_iterator，唯一的差别是此迭代器遍历一个 char 数组。

csub_match 保存一个正则表达式与一个 const char* 匹配结果的类型。可以表示整个匹配或子表达式的匹配。

默认随机数引擎 (default random engine) 用于普通用途的随机数引擎的类型别名。

格式化 IO (formatted IO) 读写操作，利用要读写的对象的类型来定义操作的行为。格式化输入操作执行适合要读取的类型的转换操作，如将 ASCII 码字符串转换为算术类型以及（默认地）忽略空白符。格式化输出操作将类型转换为可打印的字符表示形式、补白输出，还可能执行其他与输出类型相关的转换。

get 模板函数，返回给定 tuple 的指定成员。例如，get<0>(t) 返回 tuple 的第一个成员。

高位 (high-order) bitset 中下标最大的那些位。

低位 (low-order) bitset 中下标最小的那些位。